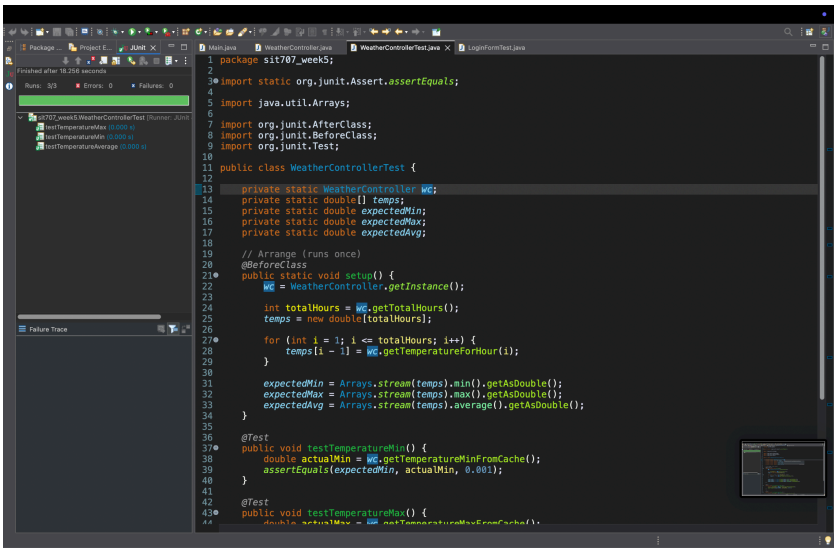
Unit test standards 1: AAA & FIRST

Task 5.1P

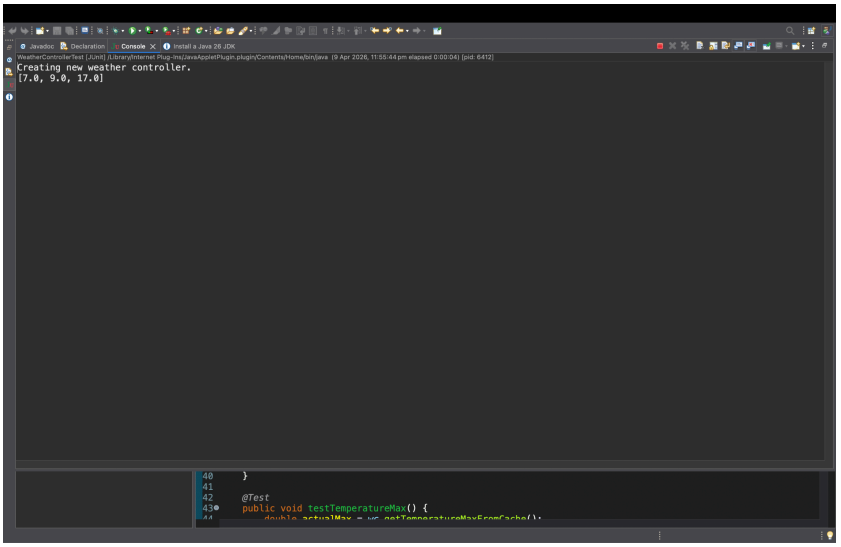
Name : Ayush Indapure
Student ID : 224880003

A screenshot of my Eclipse IDE's

(i) JUnit tab which shows test statistics including Runs, Errors and Failures



(ii) Eclipse console which shows outputs



My program's source code for tests (WeatherControllerTest.java)

```
package sit707_week5;
import static org.junit.Assert.assertEquals;
import java.util.Arrays;
import org.junit.AfterClass;
import org.junit.BeforeClass;
import org.junit.Test;
public class WeatherControllerTest {
  private static WeatherController wc;
  private static double[] temps;
  private static double expectedMin;
  private static double expectedMax;
  private static double expectedAvg;
  // Arrange (runs once)
  @BeforeClass
  public static void setup() {
      wc = WeatherController.getInstance();
      int totalHours = wc.getTotalHours();
      temps = new double[totalHours];
      for (int i = 1; i <= totalHours; i++) {
          temps[i - 1] = wc.getTemperatureForHour(i);
      }
      expectedMin = Arrays.stream(temps).min().getAsDouble();
      expectedMax =
Arrays.stream(temps).max().getAsDouble();
      expectedAvg =
Arrays.stream(temps).average().getAsDouble();
  }
  @Test
  public void testTemperatureMin() {
      double actualMin = wc.getTemperatureMinFromCache();
      assertEquals(expectedMin, actualMin, 0.001);
```

```
    }
    @Test
    public void testTemperatureMax() {
        double actualMax = wc.getTemperatureMaxFromCache();
        assertEquals(expectedMax, actualMax, 0.001);
    }
    @Test
    public void testTemperatureAverage() {
        double actualAvg =
wc.getTemperatureAverageFromCache();
        assertEquals(expectedAvg, actualAvg, 0.001);
    }
    // Cleanup (runs once)
    @AfterClass
    public static void tearDown() {
        wc.close();
    }
}
```

References(use cases)

Unit testing standards like AAA and FIRST can be hard to follow in real-world scenarios:

1. Slow operations – e.g., `getTemperatureForHour()` in `WeatherController` slows tests. *Workaround:* Initialize once and reuse data across tests - https://martinfowler.com/bliki/TestFixture.html.
2. Randomized data – random temperatures make tests non-repeatable. *Workaround:* Capture and reuse generated data or mock it.
3. Shared state – closing the controller resets cache, breaking isolation. *Workaround:* Avoid destroying shared instances during tests.
4. External dependencies – I/O or network calls slow and make tests unpredictable. *Workaround:* Use mocks or in-memory alternatives. https://testing.googleblog.com/2015/10/test-doubles-mocks-fakes-stubs.html

5. Complex setups – multiple dependent objects make arranging tests verbose. *Workaround:* Use helper methods or test fixtures for clear setup.

GITHUB : A screenshot of my GitHub page where my latest project folder is pushed
https://github.com/ayushindapure/Task-5.1P